

HOP-PAAL: A Formal Frogger 🐸 Journey

Formal Methods for Software Engineering

A.Y. 2025 – 2026

Andrea Manini and Pierluigi San Pietro

Politecnico di Milano

`firstname.lastname@polimi.it`

1 Introduction

Originally developed by Konami and published by Sega in 1981, *Frogger* represents a milestone in the history of early video game development. In this project you will focus on the Atari 2600 version¹ [8]. The primary objective of the game is to successfully guide Frogger into a home bay as many times as possible. This requires traversing a busy highway and a dangerous river, both of which are populated with frog-threatening entities. To complete a level, a player must fill five home bays by safely delivering Frogger to each bay. Successfully occupying every bay triggers the start of a new level with an incremental increase in difficulty.

In this project, you are required to formally verify the correctness of a simplified specification of the game. In particular, you must provide a model of the game as a Network of Timed Automata (NTA) using UPPAAL [2,5,9], and verify relevant properties as TCTL [1] formulae using the UPPAAL query language. This may also require extending your model to support Statistical Model Checking.

2 Game Description and Assumptions

This section describes the main entities of the game and the assumptions under which the model must be constructed. We consider a single-player version of the game at difficulty B, *i.e.*, Frogger may be safely carried off-screen over a floating object and re-enter the screen from the opposite side.

Assumption 1 *One time unit in UPPAAL corresponds to 0.1 seconds.*

Map The game is played on a map, shown in Figure 1, consisting of a grid of 18×13 cells. The figure captures the initial configuration of the game: every time a new game starts, all entities must be placed as shown. Cells can be classified by type, each governing Frogger’s safety. Yellow and grey cells are safe for Frogger, provided it is not hit by a vehicle; light blue cells are deadly: Frogger dies upon entering one; blue cells identify the home bays. Each pair of horizontally adjacent blue cells constitutes a single home bay. For completeness, an overlay of the grid onto an actual in-game screenshot is provided in Figure 2.

¹ A playable version of this game is available online at: <https://atarionline.org/atari-2600/frogger>



Floating objects The dangerous river contains three categories of floating objects: logs, turtles, and diving turtles, characterized by *size* and *speed* parameters. Floating objects in rows 8, 9, and 11 move from left to right; those in rows 7 and 10 move from right to left. Frogger may safely stand on floating objects.

Table 1: Vehicle types with their size and speed parameters.

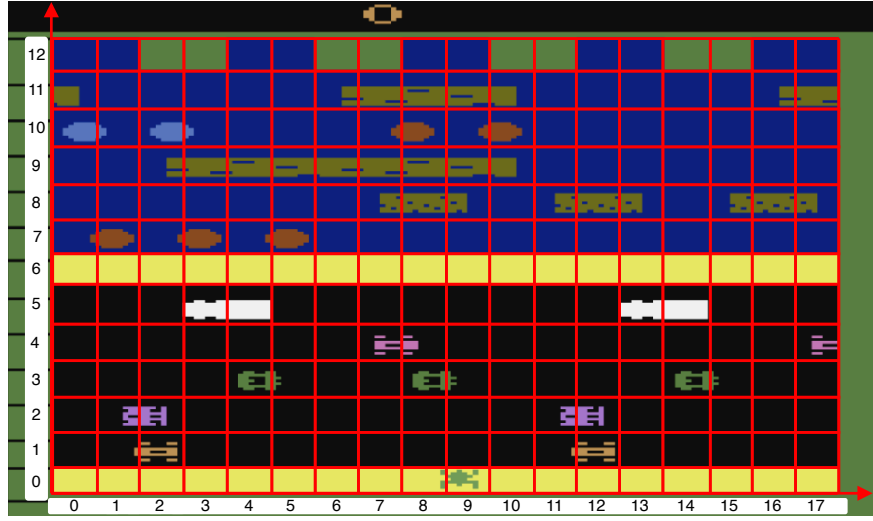


Fig. 2: An overlay of the grid onto an in-game screenshot.

- *Logs*: Frogger may safely stand on a log.
- *Turtles*: move uniformly. Frogger can move on light blue cells between turtles of the same group (a group consists of turtles separated by at most one cell).
- *Diving turtles*: behave as turtles but also cycle through four states: emerged, diving, submerged, and emerging, starting from *emerged* in the initial configuration. State transitions are determined by the following durations: emerged to diving takes 10 time units; diving to submerged happens in 5 time units; submerged to emerging requires 10 time units; and emerging back to emerged takes 5 time units. Frogger may stand on diving turtles in any state, except for *submerged*. If a diving turtle submerges while Frogger is riding it, Frogger dies. With reference to Figure 1, diving turtles are located in row 7 at cells (13,7), (15,7), and (17,7), and in row 10 at cells (9,10) and (11,10).

Whenever a floating object exits the map, it reappears on the opposite side after a delay equivalent to the time required to traverse two additional cells at its own speed. If Frogger is riding a floating object as it exits the map, it reappears on the same object when the object re-enters from the other side. The floating object types and their parameters are listed in Table 2.

Frogger It is the player-controlled character, our little, hopping hero. It may move on any cell except green and light blue ones. Remarkably, on light blue cells Frogger dies immediately unless it is riding a floating object. Indeed, Frogger may also move onto and stand on floating objects. Movement is restricted to one cell at a time in any of the cardinal directions up, down, left, or right. Frogger cannot move beyond the map boundaries. At difficulty B, Frogger may be carried to the opposite side of the map if the floating object it is riding exits the map.

Assumption 2 *Between any two consecutive moves of Frogger, at least 1 time unit must elapse; this prevents unrealistic Zeno behaviors.*

Game mechanics The game starts in the initial configuration of Figure 1. As soon as a new game starts, all vehicles and floating objects begin to move and never stop until the player loses all lives (the player is initially assigned five lives). Frogger may begin moving only after at least 2 seconds from the start, or from its last repositioning, and has only 30 seconds to reach a free home bay. Frogger cannot move into an already occupied home bay. Upon reaching a free home bay, that bay becomes occupied, Frogger is repositioned on its starting cell, and the timer resets. If Frogger dies or the timeout expires, the player loses one life, and Frogger is repositioned on its starting cell again and the timer is reset as before. In both cases, Frogger may move again after 2 seconds. Once all five home bays are occupied, a new level begins: in this case, all bays are cleared, Frogger is repositioned, and the timeout is reduced by 2 seconds. This reduction is applied at every new level, so the timeout for level $k \geq 1$ is $30 - 2(k - 1)$ seconds. The game ends when the player has no more lives. During a game, the player scores points as follows:

Moving Frogger forward (up) by one cell	1 point
Reaching a free home bay	5 points
Occupying all five home bays (completing a level)	100 points

Assumption 3 *In this project, you are not required to model lady frogs, snakes, flies, and alligators (both in the river and home bays), even though they are detailed in the manual. Furthermore, you may also avoid modeling the life-awarding mechanic as detailed in the manual (no extra lives can be granted to the player).*

3 Symbolic Version

In this version, you must consider a simplified variant of the game in which only the first level is played and Frogger can only move *up* (not down, left, or right). Every other aspect of the game must be modeled as described in Section 2.

You are required to formally model the game, without any stochastic UPPAAL features, as a NTA using the UPPAAL GUI. You decide what to model as a Timed Automata (TA) feature and what as a variable.

Properties to verify You are now required to formally express in the TCTL logic, and also verify, the following properties of your model, by creating suitable queries in the UPPAAL verifier (**one** query per property).

Floating object	Size [cells]	Speed [time units]
Turtles in row 7	1	6
Logs in row 8	3	2
Log in row 9	9	7
Turtles in row 10	1	2
Logs in row 11	5	2

Table 2: Floating objects types with their size and speed parameters.

- I) **Without considering Frogger:** the game never reaches a deadlock state.
- II) **Without considering Frogger:** all vehicles and floating objects can only move inside their predefined row.
- III) Frogger can reach the home bay in position (0,12)–(1,12).
- IV) Frogger can score 17 points reaching the home bay in position (8,12)–(9,12).
- V) Frogger can reach a yellow cell in row 6 after the player has lost two lives.
- VI) Frogger may stand on a log (with five lives) 10 seconds after the game starts.
- VII) Frogger can die riding a diving turtle that transitions to the submerged state.

4 Stochastic Version

Frogger can now move in any direction (*i.e.*, up, down, left, and right). Frogger's movement direction is governed by the following probability weights: moving up $\frac{5}{10}$, moving down $\frac{1}{10}$, moving left $\frac{2}{10}$, and moving right $\frac{2}{10}$. To ensure that Frogger attempts to move frequently enough, you *must* force Frogger to move *every* time unit, starting as soon as the 2-second countdown elapses. Furthermore, the speed of each truck is non-deterministic: the time between two consecutive moves of trucks is modeled as an exponential distribution having a rate $\lambda = \frac{1}{speed}$, where *speed* is the speed parameter reported in Table 1 for trucks.

Exponential Probability Distribution The following is an extract of [6]. The rate of exponential λ is defined as a ratio expression which specifies the rate of an exponential probability distribution. In UPPAAL, such rate expression can either be: (i) a simple integer expression; or (ii) two integer expressions separated by a colon, like $r:q$, where the rate is determined as the ratio $\lambda = r/q$.

The rate of exponential is used in statistical model checking. If a location does not have an invariant over time, then it is assumed that the probability of leaving the location is distributed according to the following exponential distribution: $Pr(\text{leaving after } t) = 1 - e^{-\lambda t}$, where $e = 2.718281828\dots$, t represents time, and λ is the fixed rate. The probability density of the exponential distribution is $\lambda e^{-\lambda t}$, where λ is the probability density of leaving at time zero, *i.e.*, as soon as some edge is enabled. The smaller the rate, the longer the delay is preferred.

Properties to verify You are now required to extend your Symbolic model and to formally express in the TCTL logic (by using UPPAAL's stochastic operators [3]) and verify the following stochastic properties of your model, by creating suitable queries in the UPPAAL verifier.

- I) Given the above specification for the Stochastic version, simulate the game for a maximum of 1000 time units and estimate the probability of completing the first level while scoring at least 200 points.
- II) Design an *alternative* movement strategy for Frogger of your own choice. You may modify the above probability weights, implement behavioural heuristics (*e.g.*, a bias towards safer rows or avoidance of submerging turtles), or define any other movement policy you find interesting. Simulate the game for $n > 0$ time units of your choice and analyze at least two interesting aspects of the game. Examples include (but are not limited to): total number of completed levels, total points scored, how many times Frogger dies touching a car, etc.

5 Delivery Instructions

It is mandatory to deliver:

- A **.pdf** report of maximum 10 pages, excluding front cover and bibliography, with no constraint on the template, describing:
 1. The model (emphasis on critical modeling choices you have made);
 2. For the Stochastic version, your Frogger's movement strategy;
 3. Verification results.
- A **.xml** file containing the Symbolic version of the game and a different **.xml** file containing the Stochastic version of the game. Make sure:
 1. Each file is the same as what you present in the report;
 2. Each file is experiment-ready, *i.e.*, it includes the queries you ran for the experiments. Queries must be ready to run with one click.

Beware: I will re-run the queries you describe in your report, hence be sure the model is experiment-ready and queries are saved in the verifier section!

Delivery deadline: also for students participating in the exchange program in UIC, is July 24th (June 20th if you want the grade registered in the first call).

WARNING: YOU MUST BE ENROLLED TO A VALID EXAM CALL TO
GET YOUR GRADE REGISTERED IN YOUR CAREER

To submit your project, send an email to andrea.manini@polimi.it including: the report, UPPAAL files, and potential supplementary material to be evaluated. Students under the UIC program must explicitly specify it in the email.

6 Minor Things (a.k.a. how to get bonus points)

Please also take into account the following when developing your model:

- ALL parameters and variables must not be hardcoded in templates or declarations, but should be generalized as much as possible (this does not apply to the queries written in the verifier).
- ALL functions created in the declarations must have meaningful comments (akin to Javadoc), *i.e.*, they must describe input parameters, output results, and give a brief overview of how the function works.
- Templates must not be graphically confusing.
- ALL templates must contain at least one significant comment which briefly explains its functioning. ALL queries must also have meaningful comments.

Grading criteria Up to 2 bonus points will be awarded to all projects with clean, well-documented code and easily readable, non-confusing templates.

- Maximum grade without stochastic features: **25** (with the bonus points, the maximum grade can increase up to **27**).
- Maximum grade with stochastic features: **30L**.
- A sufficient grade (up to **21** without considering any bonus point and up to **23** considering bonus points) is still obtainable by developing an excellent model of the Symbolic version but with no queries at all.

If some queries cannot be verified within a reasonable time on your machine, it is recommended for you to simplify the model. If the model is still too complex even after simplifications, the queries that take too long to be verified will still be evaluated based on the correctness of their specification. Whether to assess full points for those queries will be considered for each specific case. For this reason, if you encounter this issue, please document it in your final report. In this case, your report must also include details on your machine specifications (processor and RAM) on which queries were run and the maximum time limit reached before terminating the verification process.

If, after using UPPAAL, you notice that the resources of your machine are exhausting, this is due to the server process of UPPAAL running in the background. To solve, simply force kill such process.

7 Hints

To better understand how vehicles and floating objects move on the map, you may refer to the original Frogger game linked in footnote 1. In particular, whenever a vehicle or floating object exits the map, it may simultaneously be partially visible on both sides of the map at the same time if its size is greater than two cells, *i.e.*, part of the object remains visible on one side while the rest has already re-entered from the opposite side, thus producing a wraparound effect analogous to the wormhole mechanic in Pac-Man.

In the original game, a point is awarded when Frogger moves up and reaches a previously unvisited row; revisiting a row (*e.g.*, by moving backward and then forward again) does not yield additional points. In contrast, in this project every up movement increases the score by one. Moreover, in the original game, vehicles and floating objects are reset to their initial positions upon reaching a home bay. In this project, however, we assume that once the game starts these entities never stop moving (*i.e.*, they are not repositioned when a home bay is reached).

Some queries involving Frogger may be difficult, or even unfeasible, to verify (timewise) depending on your model's design. If such a query cannot be verified within a reasonable amount of time, you may justify this in the report by analyzing the factors contributing to state space explosion.² A well-argued justification can still receive full points. Notably, this does not apply to queries (I) and (II). In my solution, the symbolic queries (V) and (VI) were the most time-demanding.

Think carefully about how to model each entity; then, think about it again.

² Your model may be subject to the same limitations as those presented in [4,7].

References

1. Alur, R., Courcoubetis, C., Dill, D.: Model-checking in dense real-time. *Information and Computation* **104**(1), 2–34 (1993). <https://doi.org/10.1006/inco.1993.1024>
2. Behrmann, G., David, A., Larsen, K.G.: A Tutorial on Uppaal, pp. 200–236. Springer Berlin Heidelberg, Berlin, Heidelberg (2004), https://doi.org/10.1007/978-3-540-30080-9_7
3. David, A., Larsen, K.G., Legay, A., Mikušionis, M., Poulsen, D.B.: Uppaal smc tutorial. *Int. J. Softw. Tools Technol. Transf.* **17**(4), 397–415 (Aug 2015). <https://doi.org/10.1007/s10009-014-0361-y>, <https://doi.org/10.1007/s10009-014-0361-y>
4. Jørgensen, K.Y., Larsen, K.G., Srba, J.: Time-darts: A data structure for verification of closed timed automata. *Electronic Proceedings in Theoretical Computer Science* **102**, 141–155 (Nov 2012)
5. Larsen, K.G., Pettersson, P., Yi, W.: Uppaal (2025), <https://uppaal.org/>, accessed: 2026-03-11
6. Larsen, K.G., Pettersson, P., Yi, W.: Uppaal documentation (2025), <https://docs.uppaal.org>, accessed: 2026-02-11
7. Manini, A., Rossi, M., San Pietro, P.: Tarzan: A region-based library for forward and backward reachability of timed automata (extended version) (2026), <https://arxiv.org/abs/2602.15435>
8. Parker Brothers: Frogger: Instruction Manual (1982), https://www.gamesdatabase.org/Media/SYSTEM/Atari_2600//Manual/formated/Frogger_-_1983_-_Starpath_Corporation.pdf, for Atari 2600 & Sears Video Game Systems. Under license from Sega Enterprises, Inc.
9. UPPAAL developers: UPPAAL documentation. <https://docs.uppaal.org>, accessed: 2026-03-11